DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

EXPERIENTIAL LEARNING · PROJECT ASSESSMENT – I

Personal Expense Tracker in Python

OM

PRESENTED BY

Om Mohan Moundekar · CD25026

Under the Guidance of:
Ashwini Yerlewar

Laboratory Course:
Python Programming Lab
(N-PCCCD304P)

Session: **2026-27 (ODD)**

Autonomous Curriculum

PERSONAL EXPENSE VAULT



>_ Python 3.x • CSV Engine • Local DB

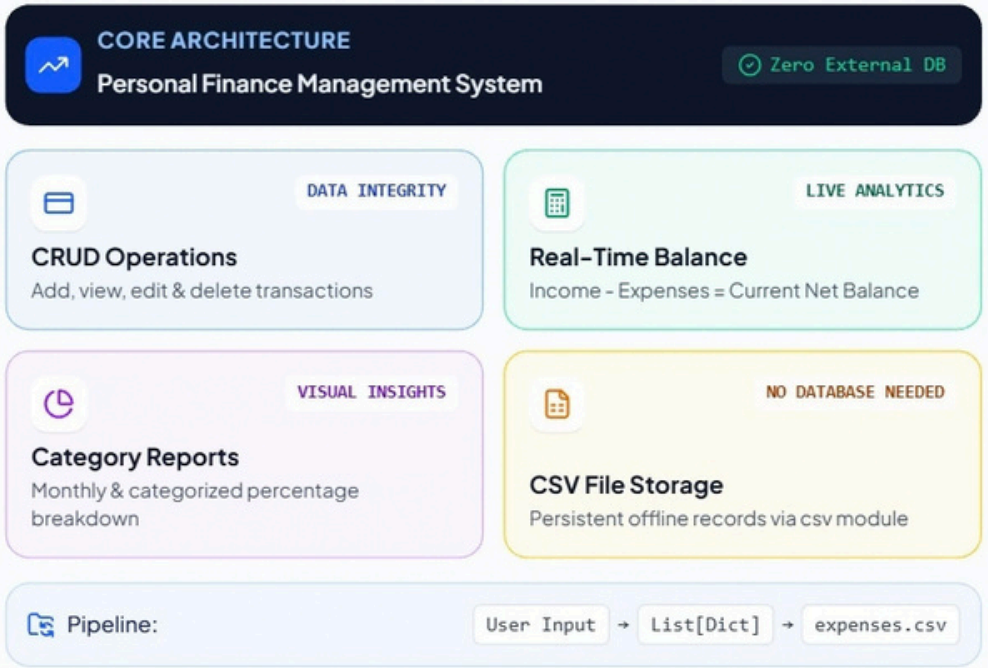
1. Problem Statement

- In daily life, individuals and students struggle to track regular expenses and maintain financial discipline.
- Manual tracking using paper notebooks or memory is time-consuming and prone to calculation mistakes.
- Without proper categorization, it is difficult to identify unnecessary expenditures and savings opportunities.
- Commercial finance apps often require internet access, have privacy concerns, and contain unnecessary complexity.
- There is a need for a simple, lightweight, and offline Python tool to record transactions and calculate balance automatically.



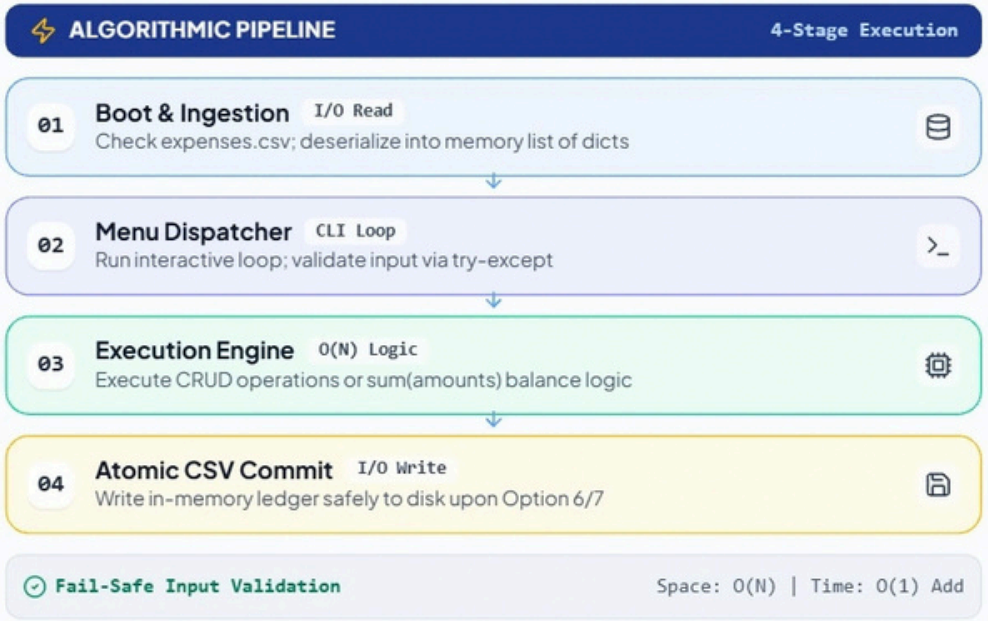
2. Objective

1. Develop an interactive, menu-driven Personal Expense Tracker using Python.
2. Allow users to record daily income credits and expenditure debits with dates and notes.
3. Enable full CRUD operations: Add, View, Update, and Delete individual financial records.
4. Organize transactions into meaningful categories (Salary, Food, Rent, Bills, Travel, Savings).
5. Compute real-time financial metrics: Total Income, Total Expenses, and Net Current Balance.
6. Generate monthly and category-wise expense breakdown reports for actionable budgeting.
7. Ensure reliable and persistent record storage using Python file handling (CSV / JSON format).
8. Practice core Python concepts: File I/O, Exception Handling, Data Structures, and Modular Design.

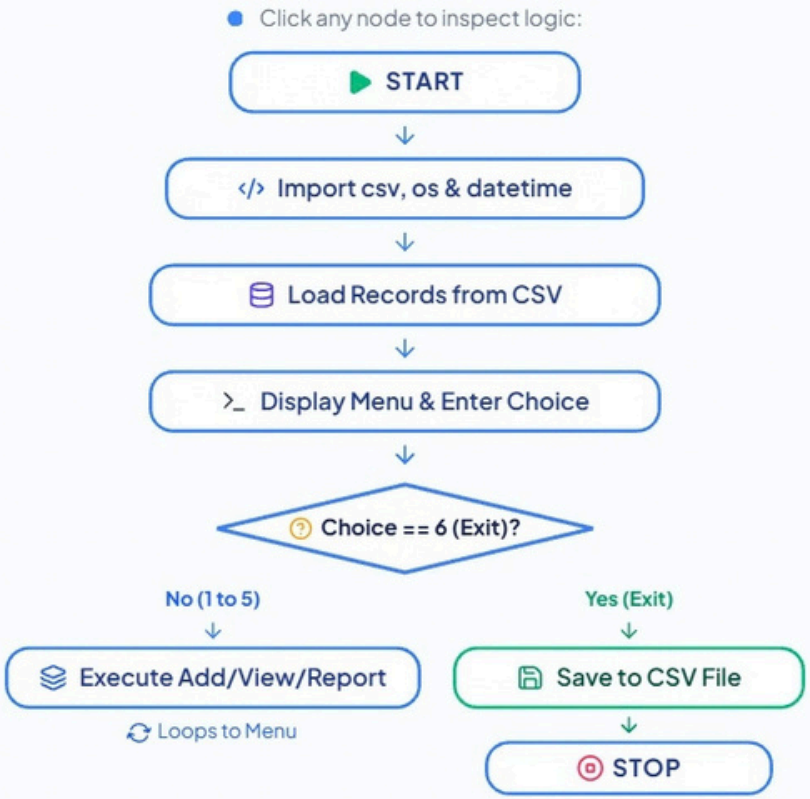


3. Algorithm

1. Start the program and define the storage file path (e.g., expenses.csv).
2. Check if the storage file exists; if present, load transactions into a list of dictionaries.
3. Display the main interactive menu with operational choices (1 to 7).
4. Accept user choice and validate numeric input using error handling.
5. If Option 1 (Add): Prompt for Type, Amount, Category, Date, and Note; append new transaction.
6. If Option 2 (View): Iterate through records and print formatted transaction history table.
7. If Option 3 / 4 (Update / Delete): Search transaction by ID; modify fields or remove record.
8. If Option 5 (Summary): Sum all income and expense values; compute Balance = Income - Expenses.
9. If Option 6 (Category Report): Aggregate expenses grouped by category; calculate percentages.
10. If Option 7 (Exit): Save all in-memory records to file, display confirmation, and stop program.



4. Flowchart



CONTROL FLOW LOGIC

Operational Loop

CRUD & analytical routines mutate in-memory list without blocking disk I/O.

Persistence Path

Option 6 triggers csv.DictWriter, writing all records safely before exit.

Active Node Detail:

Click any flowchart block on the left to inspect its role and Python implementation.

5. Understanding of Python Logic

Implementation Blueprint

1. csv & os modules handle persistent file storage, directory verification, and schema formatting.
2. datetime module automatically captures transaction timestamps and validates user dates.
3. Dictionary data structures represent individual transaction records with typed key-value pairs.
4. List of dictionaries acts as an efficient in-memory ledger database for fast indexing & traversal.
5. **List comprehensions and sum() calculate totals:** sum(t["amount"] for t in transactions if t["type"] == "expense").
6. try-except ValueError blocks prevent crashes during invalid numeric amount entries.
7. Modular functions (add_transaction, get_balance, save_data) enforce clean code separation.
8. **Core Logic:** Input & Validate → In-Memory Ledger → Aggregation Logic → Persistent Storage.

In-Memory Data Schema List[Dict]

```
record = {
  "id": 101,
  "date": "2026-09-03",
  "type": "expense", # or "income"
  "category": "College Books",
  "amount": 450.0,
  "note": "Lab manual"
}
```

load_transactions()
csv.DictReader ➔ Memory List

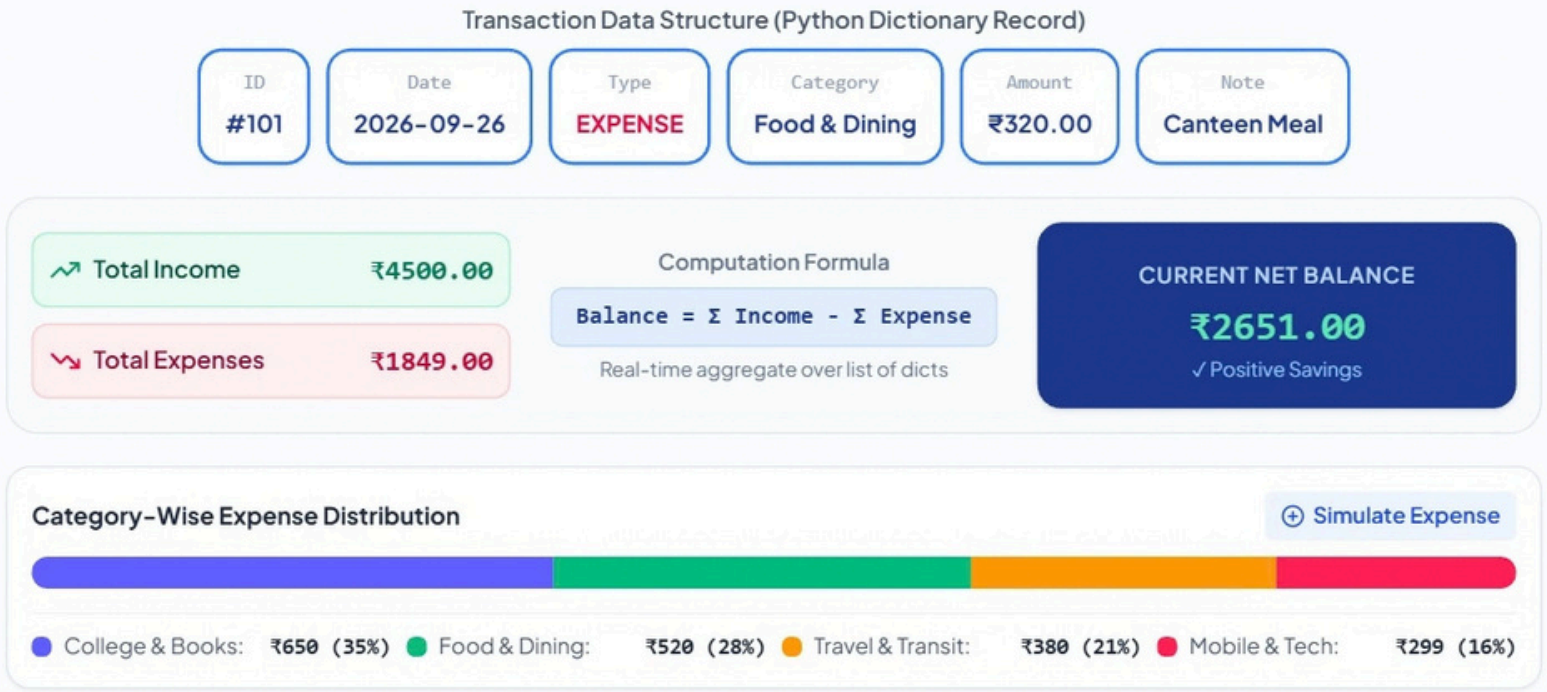
calculate_balance()
sum(income) - sum(expense)

category_report()
Accumulate % share by key

save_transactions()
csv.DictWriter atomic write

6. Graphical Representation and Working

Interactive Execution Engine



7. Python Code

Core Implementation · expense_tracker.py · Python 3.12

Standard Library · csv, os, datetime

expense_tracker.py — Clean Standalone Script

```
# Personal Expense Tracker - Concise & Complete Implementation
# Student: Ayush Nimbarte (CD25005) · Python Lab Assessment
import csv
from datetime import datetime

transactions = []

def add_transaction():
    t_type = input("Enter Type (income/expense): ").strip().lower()
    amount = float(input("Enter Amount (₹): "))
    category = input("Enter Category (e.g., Food, Travel, Books): ").strip().title()
    note = input("Enter Note: ").strip()

    record = {
        "id": len(transactions) + 101,
        "date": datetime.now().strftime("%Y-%m-%d"),
        "type": t_type,
        "category": category,
        "amount": amount,
        "note": note
    }
    transactions.append(record)
    print(f">> Recorded {t_type.upper()}: ₹{amount:.2f} (ID: {record['id']})")
```

Executable Project Code

GitHub: https://github.com/moundekarom-gif/Om-s-core.git

8. GitHub Account, Repository & Collaboration

- Create a dedicated GitHub repository for the Personal Expense Tracker project.
- Structure project files: expense_tracker.py, data/expenses.csv, and comprehensive README.md.
- Utilize Git version control to track feature development, bug fixes, and release milestones.
- **Repository Link:** https://github.com/moundekarom-gif/Om-s-core/tree/main
- Future collaboration includes GUI migration (Tkinter/PyQt), SQLite database, and cloud sync.



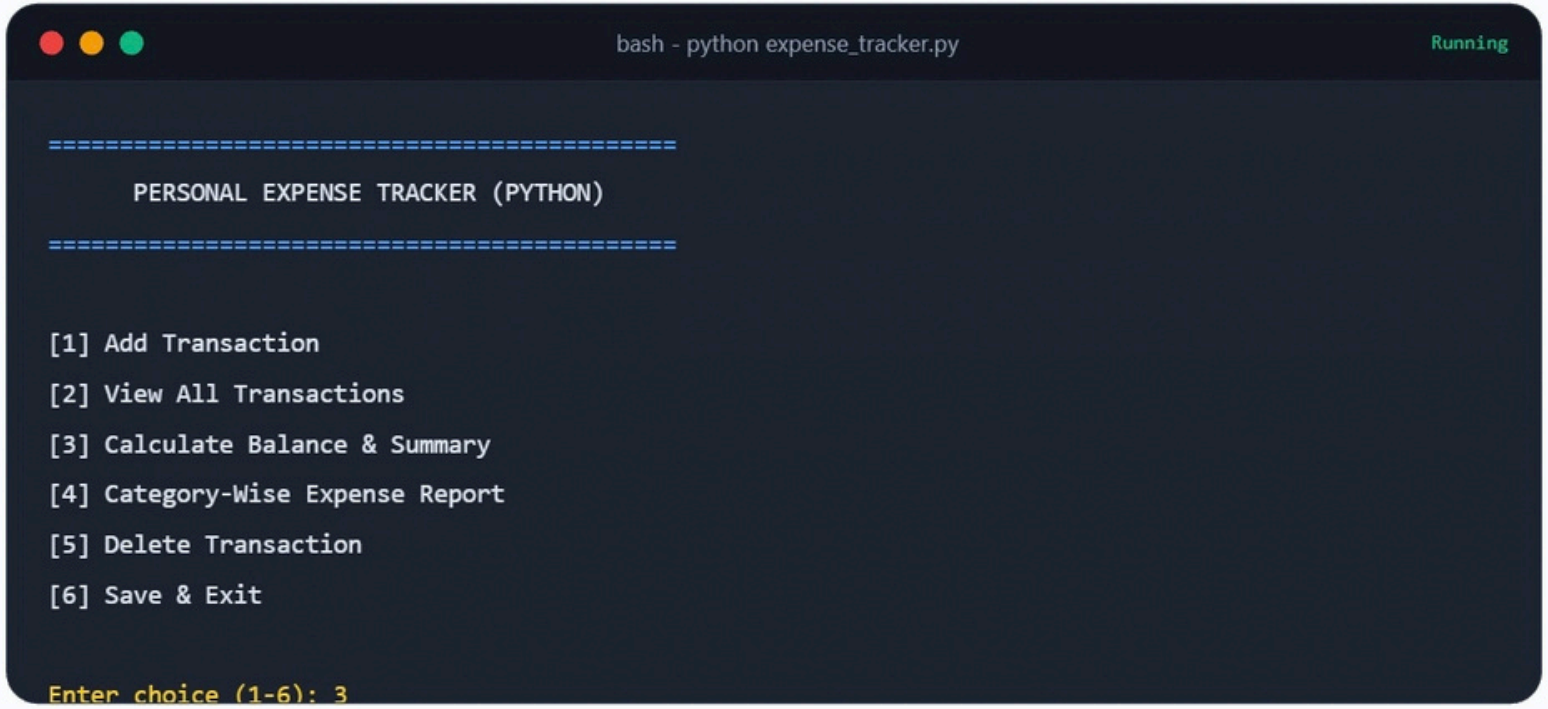
Click any step to inspect Git command and lifecycle role

9. Result / Expected Output

Live Console Output

>_ Python CLI Environment

Switch to Live CLI



10. Conclusion

- The Python Personal Expense Tracker provides an efficient, lightweight system for daily finance management.
- Successfully implements complete CRUD capabilities with intuitive menu-driven navigation.
- File handling guarantees data persistence across sessions without requiring external database servers.
- Real-time aggregation accurately computes Total Income, Total Expenses, and Net Balance.
- Category-wise reporting empowers users with clear spending insights to improve financial discipline.
- Demonstrates practical mastery of Python loops, conditional branching, file I/O, and data structures.

EXPERIENTIAL LEARNING · ASSESSMENT-I

Thank You!

Ready for Technical Evaluation & Viva Voce Q&A

✓ 100% Core Requirements Satisfied

Guide: Ashwini Yerlewar